Parallelizing GPT-2 -"It works on my GPU"

CSCI 563

Siggy Sigler, Sam Abderholden, Dawson Matthews

Colorado School of Mines

May 4, 2026

Introduction

Parallelizing the forward pass of MicroGPT.

- MicroGPT is a 200 line, dependency free, Python implementation of GPT-2 created by Andrej Karpathty
- Parallelizing ONLY the forward pass of MicroGPT
  1. **Tokenization & Embedding:** Mapping inputs to high-dimensional space.
  2. **Self-Attention:** Parallelizing the Q, K, V matrix operations.
  3. **Loss Calculation:** Efficiently computing cross-entropy across batches.
- Implement batching to improve parallel performance

Serial Implementation

`name_list: [[BOS, E, L, L, A, BOS], [name_B], [name_C]]`

`BOS = Beginning of Sequence`

Serial

```
for name in name_list:
  for token in name:
    embedding(token)
    for layer in layers:
      attention(token)
      mlp(token)
    calc_loss(token)
sum_loss(tokens)
```

Serial Analysis

Big-O:

- T = sequence length, E = embedding size, L = layer, N = names
- $O(N \cdot L \cdot (C_{\text{attn}} + C_{\text{mlp}}))$
- $O(C_{\text{attn}}) = T^2 \cdot E$
- $O(C_{\text{mlp}}) = T \cdot E^2$
- Since names are mostly constant size it becomes $\approx O(N \cdot L \cdot E^2)$

Memory vs CPU Bound:

- Total Time: 6.41 s, CPU Time: 6.19
- CPU utilization $\approx \frac{6.19}{6.41} \approx 96.6\% \Rightarrow$ CPU-bound

Static Analysis:

- **99.4%** of work spent doing matrix mult, (0.4% for rmnsnorm, SM, emb)
- **71.1%** for MLP **28.3%** for ATTN
- By Amdahl's Law, with $p = 0.998$, the theoretical maximum speedup is $\frac{1}{1-p} \approx 500\times$

Serial vs. Parallel Implementation

`name_list: [[BOS, E, L, L, A, BOS], [name_B], [name_C]]`

`BOS = Beginning of Sequence`

Serial

```
for name in name_list:
    for token in name:
        embedding(token)
        for layer in layers:
            attention(token)
            mlp(token)
        calc_loss(token)
sum_loss(tokens)
```

Parallel (per batch)

```
calculate_embeddings(name_list)

for layer in layers:
    calculate_attention_scores(
    name_list)
    calculate_next_hidden_state(
    name_list)

calc_loss(name_list)
```

Parallelization Strategy

- **Batching**
  - For each sequence launch a transformer $\rightarrow$ run transformer on all sequences at once
- Store all of our information in a flattened array
  - Allows assignment of work easily by batch to threads
- Store a "workspace" on the GPU that holds weights, hidden states, etc.
- Large portions of the forward pass are parallelizable
  - **Embedding calculations**
  - **RMS normalization and linear calculations**
  - **Attention**
- **Layers must be performed sequentially**

Performance Evaluation

	Python Serial	**C++ Serial**	**CUDA Parallel**
Names: 200 Layers: 1 Embeddings: 64	44.68 seconds	0.0644 seconds	0.114 seconds
Names: 1,000 Layers: 2 Embeddings: 128	1721.97 seconds (28.7 minutes)	2.16 seconds	0.158 seconds
Names: 10,000 Layers: 8 Embeddings: 512	Skipped (Too long)	1,583.08 seconds (26.4 minutes)	3.52 seconds